

have a list of only moves that are possible in this particular game. You can run `print(allpossible)` to see all of these moves and check that they're correct.

Now that we have a list of every possible move, we can construct the game tree. We'll record a game tree as a nested list of moves. Remember that each move can be recorded as a list of ordered pairs, like `[(4,4),(4,3)]`, the first move in the left branch of [Figure 10-3](#). If we wanted to express a tree that consisted of only the top two moves in [Figure 10-3](#), we could write it as follows:

```
simple_tree = [[(4,4),(4,3)],[(1,3),(2,3)]]
```

This tree contains only two moves: the ones we're considering playing in the current state of the game in [Figure 10-3](#). If we want to include the opponent's potential responses, we'll have to add another layer of nesting. We do this by putting each move in a list together with its *children*, the moves that branch out from the original move. Let's start by adding empty lists representing a move's children:

```
simple_tree_with_children = [[[(4,4),(4,3)],[]],[(1,3),(2,3)],[]]
```

Take a moment to make sure you see all the nesting we've done. Each move is a list itself, as well as the first element of a list that will also contain the list's children. Then, all of those lists together are stored in a master list that is our full tree.

We can express the entire game tree from [Figure 10-3](#), including the opponent's responses, with this nested list structure:

```
full_tree = [[[(4,4),(4,3)],[(1,3),(2,3)],[(3,1),(4,1)]]],[(1,3),(2,3)],  
[[[(4,4),(4,3)],[(3,1),(4,1)]]]]
```

The square brackets quickly get unwieldy, but we need the nested structure so we can correctly keep track of which moves are which moves' children.

Instead of writing out game trees manually, we can build a function that will create them for us. It will take our list of possible moves as an input and then append each move to the tree ([Listing 10-4](#)).